CHAPTER 1 : DATA DRIVEN TESTING

Types of Test Data in Automation

Common Data

- The Data which is common for all the test scripts/applications/projects is called as common data
- Example: url, usernames, passwords
- Usually stored in a file like: CommonData.properties.

Test Script Data

- The Data which is common for particular test scripts/applications/projects is called as Test script data
- Example: product names, Organization names
- Usually stored in a file like: testScriptData.xlsx.

Why DDT?

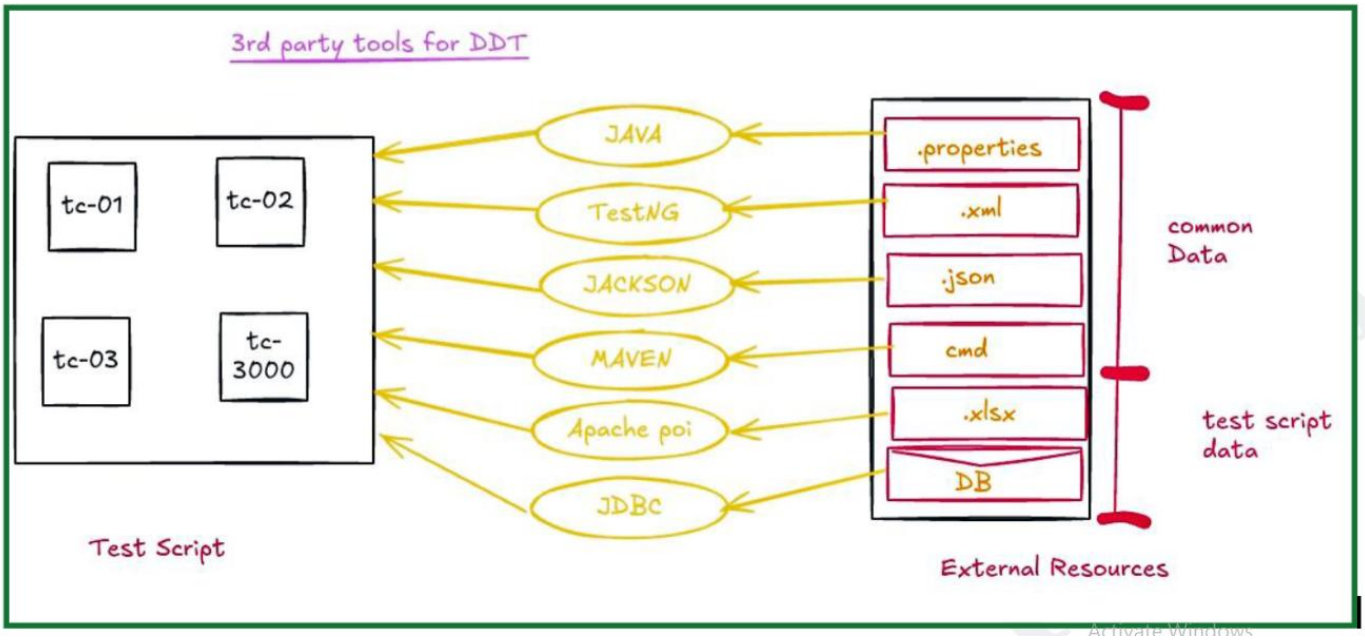
- The first rule of automation, we should never ever hardcode the test data into our scripts.
- Hardcoding test data makes maintenance and updates difficult.
- DDT helps separate test logic from test data, improving reusability and flexibility.

What is Data Driven Testing?

- Testing the application with the help of external resources and running the test scripts is called as DDT.

Advantages of DDT

1. Maintenance is easy
2. Modification is easy
3. Cross browser testing
4. Multi environment testing
5. Testing with multiple user credentials.
6. Re-running the same test with different inputs is easy.
7. Perfect for Agile methodology, where frequent builds require to run daily with different data sets.



Data Driven Testing Using Properties File

1. What is a Properties File?

- A Properties File is a Java-supported file used to store key-value pairs.
- Commonly used to store configuration or test data (common data).

1.1. What are the rules to create Properties File:

#1. Data should be in key value format and in compulsorily String
#2. key value should be separated with space or = or :
#3. One pair per line
#4. Keys should be UNIQUE
#5. Extension must be .properties

url https://localhost:8888
bro chrome
un admin
pwd manager

2. Why do we Use a Properties File?

1. Lightweight and easy to maintain.
2. Faster to read than Excel or other formats.
   Java provides a built-in Properties class for reading these files.

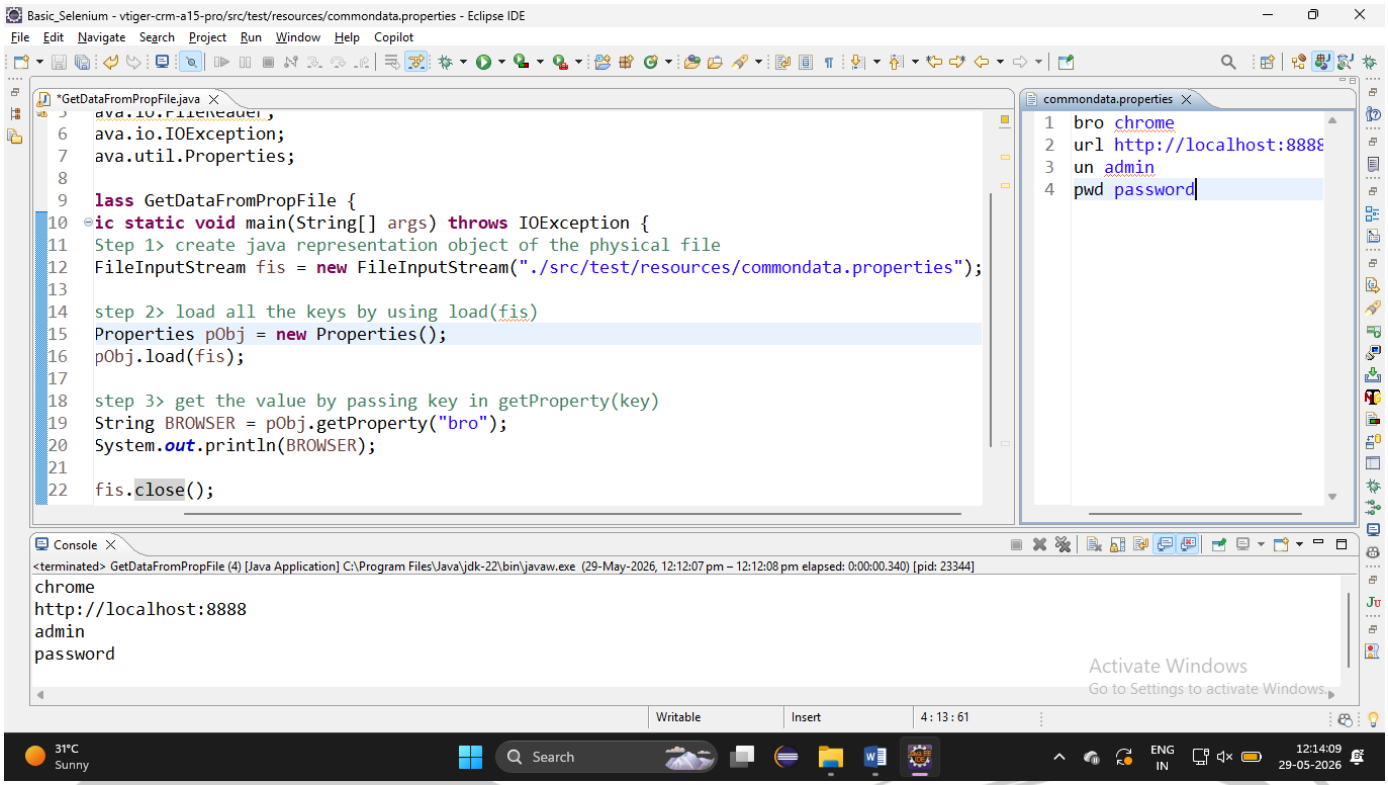
3. How to Read Data from a Properties File in Java?

```
//          step 1> Create a java representation object of the physical file
//          /vtiger-crm-framework-m5/src/test/resources/commondata.properties
//          modified to
//          ./src/test/resources/commondata.properties
            FileInputStream fis = new FileInputStream("./src/test/resources/commondata.properties");

//          Properties class
//          step 2> by using load(), load all the keys
            Properties pObj = new Properties();
            pObj.load(fis);

//          step 3> by using getProperty("key"), get the value
            String value = pObj.getProperty("key");
            System.out.println(value);
```

📌 Note:
Properties file is faster and more efficient for reading data compared to Excel.



Feature	File	FileReader	FileInputStream
Represents file path	✔	✘	✘
Reads data	✘	✔	✔
Reads characters	✘	✔	✘
Reads bytes	✘	✘	✔
Good for text files	✘	✔	✔
Good for binary files	✘	✘	✔
Used in Selenium frameworks	Sometimes	Very common	Very common

Reading Data from JSON File

What is JSON?

JSON = **JavaScript Object Notation**

It is a lightweight format used to store and exchange data.

Example:

```
{
  "username": "admin",
  "password": "admin123"
}
```

Advantages

Benefit	Explanation
External Test Data	No hardcoding
Reusable	Same data for multiple tests
Easy Maintenance	Update JSON only
Framework Standard	Used in real projects
Supports Nested Data	Complex test scenarios

What is Simple JSON?

`simple-json` is a lightweight Java library used to:

- Read JSON
- Parse JSON
- Write JSON

Library Name:

```
json-simple
```

Maven Dependency

Add this in `pom.xml`

```
<dependency>
    <groupId>com.googlecode.json-simple</groupId>
    <artifactId>json-simple</artifactId>
    <version>1.1.1</version>
</dependency>
```

Create JSON File

File Name:

testData.json

Inside JSON File

```
{
  "username": "admin",
  "password": "password",
  "url": "https://localhost:8888"
}
```

Read JSON File in Java

Full Program

```
package jsonPractice;

import java.io.FileReader;

import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;

public class ReadJsonData {

    public static void main(String[] args) throws Exception {

        // Step 1: Create JSONParser object
        JSONParser parser = new JSONParser();

        // Step 2: Pass JSON file path
        FileReader reader = new FileReader(
                "./src/test/resources/testData.json");

        // Step 3: Parse JSON file
        Object obj = parser.parse(reader);

        // Step 4: Convert object into JSONObject
        JSONObject jsonObject = (JSONObject) obj;

        // Step 5: Read data using key
        String username = (String) jsonObject.get("username");
        String password = (String) jsonObject.get("password");
        String url = (String) jsonObject.get("url");

        // Step 6: Print data
        System.out.println(username);
        System.out.println(password);
        System.out.println(url);
    }
}
```

Output

```
admin
password
https://localhost:8888
```

Interview Questions

Q1. Why JSON is used in Selenium?

Answer:

```
JSON is used for external test data management.
It improves framework maintainability and avoids hardcoding.
```

Q2. Difference between JSON and Excel?

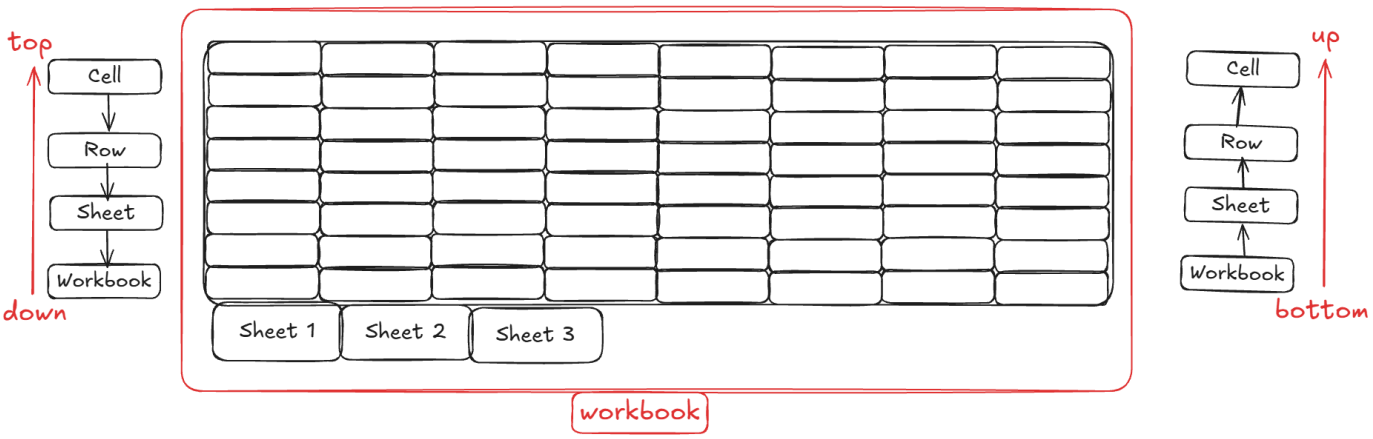
JSON	Excel
Lightweight	Heavy
Faster	Slower
Better for APIs	Better for manual viewing
Structured	Tabular

Q3. What is JSONObject?

Answer:

```
JSONObject is a class used to represent JSON data in key-value pair format.
```

Data Driven Testing Using Excel File (Apache POI)



Apache POI for Excel Handling

What is Apache POI?

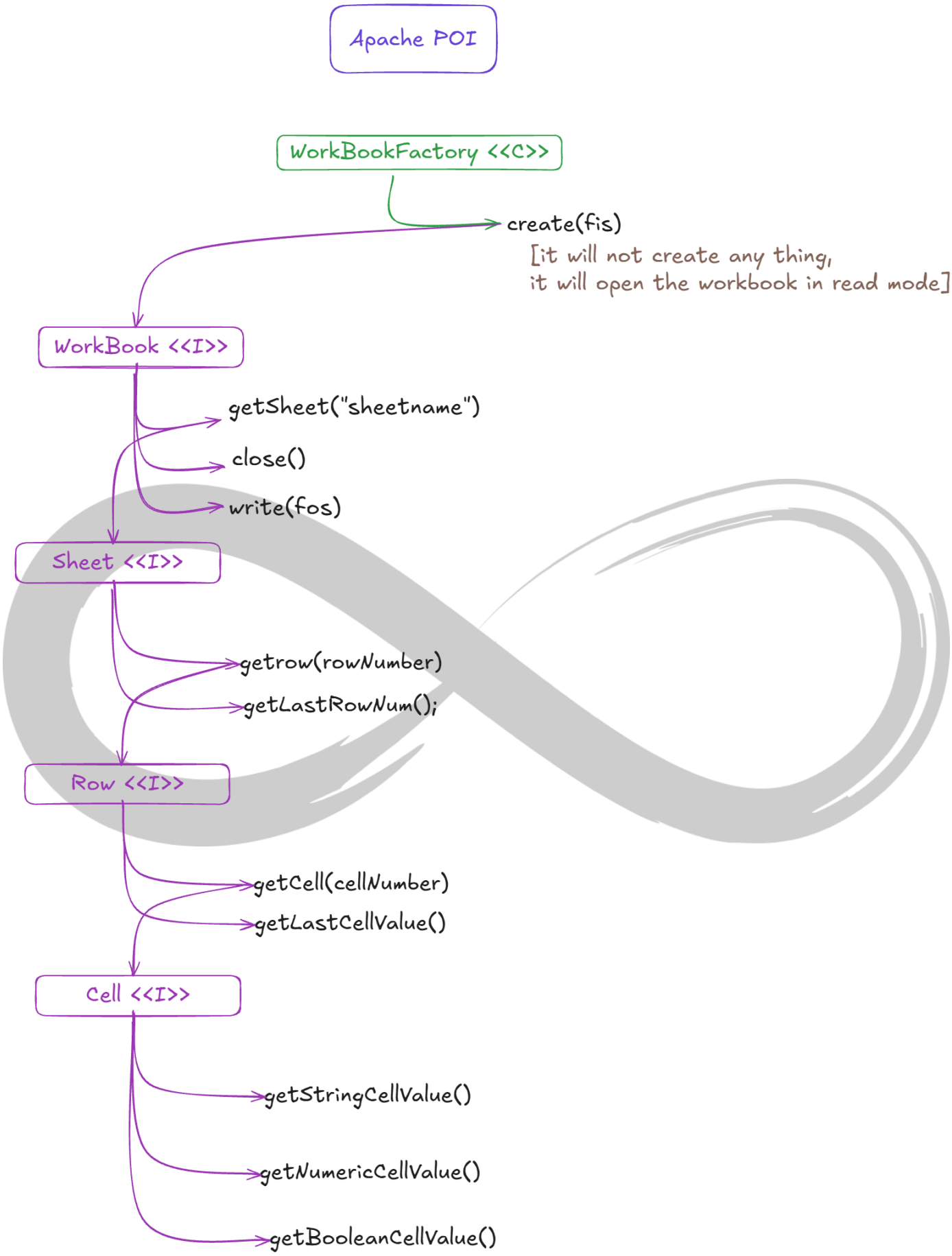
- Apache POI is an open-source Java library that allows reading and writing of Microsoft Office formats.
- It supports formats like:
  - .xls and .xlsx (Excel)
  - .docx (Word)
  - .pptx (PowerPoint)
- In real-world automation projects, companies prefer to use Excel for test data due to its structured tabular format.

Apache POI Setup (Maven)

- Open your Maven Project in IDE.
- Open the pom.xml file.
- Visit: https://mvnrepository.com
- Search for: apache poi
- Add the following dependencies inside <dependencies> tag:

```
<dependency>
   <groupId>org.apache.poi</groupId>
   <artifactId>poi</artifactId>
   <version>5.5.1</version>
</dependency>

<dependency>
   <groupId>org.apache.poi</groupId>
   <artifactId>poi-ooxml</artifactId>
   <version>5.5.1</version>
</dependency>
```


Apache POI
WorkBookFactory <<C>>
create(fis)
[it will not create any thing,
it will open the workbook in read mode]
WorkBook <<I>>
getSheet("sheetname")
close()
write(fos)
Sheet <<I>>
getrow(rowNumber)
getLastRowNum();
Row <<I>>
getCell(cellNumber)
getLastCellValue()
Cell <<I>>
getStringCellValue()
getNumericCellValue()
getBooleanCellValue()

Get the data from excel file

```
//          step 1> get the java representation object of the physical file
//          /vtiger-crm-framework-m5/src/test/resources/testScriptData.xlsx
//          modified to
//          ./src/test/resources/testScriptData.xlsx
            FileInputStream fis = new FileInputStream("./src/test/resources/testScriptData.xlsx");

//          step 2> get the access of workbook
            Workbook wb = WorkbookFactory.create(fis);

//          step 3> get the access of sheet
            Sheet sheet = wb.getSheet("dummy");

//          step 4> get the access of row
            Row row = sheet.getRow(7);

//          step 5> get the access of cell
            Cell cell = row.getCell(0);

//          step 6> get the data
            String data = cell.getStringCellValue();
//          cell.getNumericCellValue();
//          cell.getBooleanCellValue();
            System.out.println(data);

//          don't forget to close the workbook
            wb.close();
```